

Contents

1	Introduction	1
1.1	Why Native Re-Implementation?	3
2	Related Work	3
3	Methodology	5
3.1	Port Construction	6
3.2	Validation	7
3.3	Experimental Machine and Configurations	7
3.4	Heap Selection	9
3.5	Scope and Limitations	9
4	Native Benchmark Ports	9
5	Results	12
5.1	RQ1: How do modern Java collectors compare with native baselines?	13
5.2	RQ2: Does giving Java more heap space close the gap?	16
5.3	RQ3: How do Java and native execution differ in memory footprint?	17
5.4	RQ4: What microarchitectural differences accompany the runtime gap?	21
5.5	RQ5: How much does collector choice matter?	22
5.6	RQ6: What do the C++ ports reveal about implementation structure?	23
5.7	RQ7: How sensitive is the observed Java/native gap to optimization of the native baseline?	25
6	Discussion	27
6.1	Abnormal and Methodologically Conditional Cases	29
6.2	Limitations	30
7	Conclusion	30

8	Artifacts/Repo	31
9	References	32
10	Appendix	35
10.1	biojava Extra Figures	35
10.2	graphchi Extra Figures	37
10.3	h2 Extra Figures	38
10.4	lusearch Extra Figures	40
10.5	sunflow Extra Figures	42
10.6	xalan Extra Figures	44
10.7	zxing Extra Figures	46

1 Introduction

Garbage collection (GC) is a defining feature of managed runtime systems such as Java. By automating memory reclamation, GC relieves programmers of the burden of precisely managing their own memory, allowing them to avoid memory-management bugs. However, these benefits come with known costs: GC consumes execution time, often requires additional memory headroom, and can alter cache behavior, locality, and pause-time characteristics. The main cost of GC is the overhead of garbage collection itself. Explicit memory management (EMM), by contrast, offers direct control over allocation and reclamation, but shifts complexity and correctness burdens back to the programmer.

A quantitative study aimed at understanding the GC vs. EMM tradeoff is Hertz and Berger’s 2005 OOPSLA paper [1]. To avoid comparing different source languages or differently structured programs, they introduced an *oracular* memory manager that executed unmodified Java programs as if they used explicit memory management. Their results provided a key insight: GC was competitive with explicit memory management only when given substantially more memory, matching or slightly exceeding EMM when given roughly 5× the required memory, slowing by about 17% at 3×, and by about 70% at smaller heap sizes on average [1]. They further showed that explicit memory management generally achieved better memory utilization and page-level locality [1]. Their results have had a lasting influence on how researchers and practitioners think about the performance costs of automatic memory management, and the debate continues to this day.

Twenty years later, however, it is no longer clear that the same conclusions hold for modern systems. The collector landscape has changed substantially. Contemporary JVMs now include mature region-based and highly concurrent collectors such as G1 and ZGC [8, 18, 19]. These collectors shift work concurrently and generationally to reduce pause times, but doing so introduces new trade-offs in barrier costs, metadata, and CPU overhead [8]. At the same time, modern work suggests that GC costs are more multidimensional. Cai et al. [3] show that, even in OpenJDK 17, production collectors can still incur substantial overhead relative to a distilled no-GC baseline, with reported lower bounds of 7–82% more wall-clock time and 6–92% more CPU cycles at a modest

heap size. Sareen and Blackburn [9] likewise show that the modern picture is more nuanced than a simple “GC is slow.” In modern allocators, GC can have little impact on mutator time in some settings, while space overhead varies substantially by collector design, from roughly 65–80% for simple copying collectors to 11–17% for Immix, relative to a baseline approximating manual memory management. In addition, more up-to-date benchmark suites such as DaCapo provide a new baseline against which to compare earlier results.

There have also been improvements on the EMM side. Hertz and Berger were compared against a general-purpose allocator like LEA, which was appropriate at the time, but current systems frequently rely on highly engineered general-purpose allocators such as jemalloc and mimalloc. Each is designed to improve scalability, reduce contention, and enhance page locality [11, 13, 14, 15].

This paper revisits the GC-versus-EMM question using a different methodology. Rather than attempting to retrofit explicit reclamation into a contemporary JVM, we compare modern Java executions against workload-equivalent native re-implementations of selected DaCapo benchmarks in C. We focus on the DaCapo 23.11 workloads as a more modern benchmark corpus. This choice is intended to preserve algorithmic and workload comparability while avoiding many of the validity threats that a JVM-internal oracle replay may introduce, as previous work argues against [3]. Our goal is not to claim a pure isolation of GC cost in the strong sense achieved by the original oracle experiment. Instead, we ask a narrower but still important modern question: under contemporary collectors, allocators, workloads, and hardware, how closely do Java executions approach comparable native baselines?

To answer that question, the paper is organized around seven research questions:

RQ1. How do modern Java collectors compare with native baselines in overall runtime?

RQ2. Does giving Java more heap space close the performance gap to native execution?

RQ3. How do Java and native executions differ in resident memory footprint and dynamic work?

RQ4. What microarchitectural differences accompany the observed runtime gap?

RQ5. How much does collector choice matter once Java is tuned fairly?

RQ6. What do the C++ probe implementations reveal about the role of implementation structure?

RQ7. How sensitive is the observed Java/native gap to optimization of the native baseline?

1.1 Why Native Re-Implementation?

Two practical issues motivated our shift away from oracular replay. First, implementation fragility. Replacing a GC-managed heap with `malloc/free` inside a modern JVM requires interacting with compressed OOPs, TLABs, klass pointers, mark-word manipulation, and JIT-generated fast paths. In our own attempts with a modified Corretto-25 JVM, these interactions made the system difficult to stabilize and led to failures, including use-after-free errors. Second, oracle construction is expensive and approximate. Our tracing experiments with Elephant Tracks [20] showed severe execution slowdown. In addition, extensive configuration was required for both the trace and replay runs to produce a valid measurement. However, the results then became difficult to measure fairly.

We therefore use native ports generated via LLMs, not because the approach is identical to the original oracle experiment, but because it provides a practical way to compare modern Java executions against benchmark-equivalent C and C++ native baselines. This comparison does not measure the isolated cost of GC in the strongest possible way, since language implementation and program structure are not held perfectly constant. It does, however, allow us to evaluate whether tuned modern Java executions approach the performance of comparable native implementations.

2 Related Work

The central study that informs our research is Hertz and Berger’s 2005 study on garbage collection versus explicit memory management [1]. Their key contribution was to introduce an oracular

memory manager that enabled Java programs to execute with `malloc/free` semantics under oracle guidance. Their results showed a clear space–time tradeoff and served as the main baseline for our work. However, to understand what has changed, we must examine more recent work.

A study by Cai et al. [3] suggests that the collector side of the original study has changed substantially. Cai et al. [3] argue that the absolute cost of GC remains insufficiently understood and that the Hertz & Berger methodology does not transfer cleanly to modern production runtimes. Using Epsilon as a no-op baseline, they develop a methodology that places a lower bound on GC overhead in OpenJDK 17 and show that modern production collectors still incur substantial costs. Across a diverse suite of modern benchmarks and at a modest heap size, they estimate that production GCs incur at least 7–82% more wall-clock time and 6–92% more CPU cycles than the distilled baseline. Averaged over 16 benchmarks at $3.0\times$ heap, their lower-bound overheads are 1.13, 1.06, 1.09, 1.43, and 1.54 in time for Serial, Parallel, G1, Shenandoah, and ZGC, respectively, and 1.06, 1.10, 1.14, 1.47, and 1.68 in CPU cycles. Research on modern collectors provides some support for our own reexamination of such collectors to determine whether giving more heap space improves performance relative to an explicit allocator.

Zhao and Blackburn [8] deconstruct a modern collector, G1, into its major mechanisms and quantify the costs and benefits of each. Their analysis shows that modern GCs hide multiple underlying tradeoffs in concurrency, barriers, remembered sets, and evacuation strategy. In particular, they find that concurrent marking and generational collection reduce 95th-percentile GC pause times by 64% and 93%, showing how much modern pause-time improvements depend on concurrency and generation-aware collection. These results are relevant to our study because they show that modern GC performance reflects a bundle of design choices in concurrency, barriers, remembered sets, and evacuation strategies.

Sareen and Blackburn [9] broaden the picture further by re-examining the costs and benefits of automatic memory management using modern allocator methodologies. Their results show that space overhead is highly collector-dependent and that some commonly repeated interpretations of Hertz and Berger’s space-overhead result overstate what that paper directly established. Their

analysis matters directly to our study because it suggests that a modern revisit requires us to examine collector overhead, which varies substantially by design. Thus, a fair study must use modern collectors and current explicit allocators.

Carpen-Amarie et al. [6] add an important cache-level perspective, showing that concurrent GC can cause severe cache disruption in cache-sensitive workloads but only modest effects on most modern server benchmarks. Their discovery is important to us because it means that some benchmarks may expose substantial cache and concurrency penalties, while others may largely hide them. Therefore, our benchmark ports must be workload-equivalent and must examine results beyond runtime alone.

Taken together, prior work leaves open whether modern Java and modern workloads still exhibit the same space–time tradeoff that has been observed before under a new cross-language comparison using native ports.

3 Methodology

We revisit the GC vs. EMM question using a native re-implementation methodology rather than the oracle-guided replay used by Hertz and Berger [1]. We aim to compare modern Java executions against workload-equivalent native ports of eight DaCapo 23.11 workloads. DaCapo is a Java benchmark suite designed for research use and consists of real-world Java applications with non-trivial memory loads [4]. A point to mention is that many of the DaCapo benchmarks are thin wrappers around larger upstream systems. So we adjusted accordingly: our porting target was not just the wrapper, but any large, reachable library the benchmark used. Thus, we sought to create a program with a workload similar to the benchmark rather than a simple implementation. However, our new methodology does not preserve the exact JVM allocation sites, runtime services, or compiler pipelines of managed execution, so the resulting comparison should not be read as a pure isolation of GC vs. EMM as the original methodology allowed. Instead, our methodology provides a modern comparison between Java and native ports designed to preserve the same workload and

structure as closely as possible.

3.1 Port Construction

The native benchmark suite was built through an LLM-assisted workflow. A distinction to make is that most of the benchmarks ported were in C, not C++. We produced more C ports than C++ ports because the C ports were faster to implement, allowing us to cover more benchmarks. Because the C++ ports took longer to implement, there are fewer of them than the C ports. The role of the C++ ports was not to simply change the language of the benchmark, but to restore a more object-oriented structure, reduce ownership overhead, and adopt a library-like organization, even when a compact C port might otherwise simplify important parts of the original Java code.

For each of our chosen benchmarks, we first wrote a design specification describing the DaCapo workload, the upstream Java path (if applicable), the intended C or C++ architecture, the allocation behavior we wanted to preserve, and the validation checks to be performed before any measurements. These plans were then converted into implementation plans for a coding agent. Much of this work was done via Claude Code, using the Opus 4.6 model in a high-thinking configuration. The AI assistant was the main worker on porting the Java source code, while the researcher acted more as a system architect on design decisions.

As mentioned, the workflow had four stages. First, a source-discovery stage identified the actual upstream library and code that executed the DaCapo benchmark. Second, a design stage was used to guide the native architecture before an implementation plan was developed. Third, there was an implementation stage in which the port could break the large design into smaller parts for testing and validation. For the C implementations, this was more module-to-module, whereas for C++, it was more class-by-class. Fourth, a validation stage required the port to first prove that it reproduced the Java benchmark’s behavior. That validation stage compared the port against Java-generated reference outputs, not just whether it compiled or ran. We gave the model more structure so we could guide the LLM toward a more faithful port rather than merely asking it to produce a C port of a Java workload.

3.2 Validation

Validation was layered and benchmark-specific. At the lower levels, ports were checked with unit tests for parsers, data structures, numerical kernels, and other helper routines. At the workload level, each benchmark was validated against Java reference behavior using one or more of the following: golden files, byte-identical outputs, digest comparisons, structured equivalence checks, deterministic replay, and per-input oracle tables. Where possible, we also collected allocation-pressure reports to determine whether an output-valid port was still doing the same amount of work.

The main goal was to ensure that ported benchmarks were not merely plausibly equivalent but properly faithful ports, so we could run fair comparisons. However not all ports are perfect representations of their equivalent Java ports. So we do not aim to treat all the ports as equally strong evidence. Instead, we aim to interpret a result’s weakness as part of a broader trend across the benchmarks, then examine each empirical result with greater confidence.

3.3 Experimental Machine and Configurations

The GC side of the study uses DaCapo 23.11-Chopin/MR2 as the benchmark basis. The DaCapo harness supports multiple executions, with the first $n - 1$ iterations serving as a warmup and the final iteration serving as the timed iteration [4]. DaCapo supports repeated same-process iterations, but the Java reruns in our study were executed as fresh processes with single-iteration runs and a single benchmark workload thread using the flags `-n 1` and `-t 1`. We used these flags in order to avoid measuring peak steady-state behavior that a JVM can reach through repeated runs of similar allocation workloads. Prior work shows that VM benchmarks often fail to reach a consistent steady-state peak performance, and that static warm-up estimates are frequently inaccurate [25, 26]. So we see no reason to add an arbitrary fixed warmup that would have changed the targeted measurements without a guarantee of improving the results, while also making the results harder to parse, since all iterations of a run would have been done in the same process.

All measurements were collected on a remote Linux host running Ubuntu 24.04 with Linux ker-

nel 6.17.0-1012-aws on an aarch64 system. The machine exposes 16 logical CPUs, ARM Cortex-A72 cores, with 31 GiB of RAM and no swap configured. Java experiments used Ubuntu OpenJDK 21.0.10, while native binaries were compiled with GCC 13.3.0 and G++ 13.3.0. Hardware-counter collection used `perf` 6.17.13. One limitation of the setup is that the harness logs do not record the exact AWS instance type, nor do they record a language-runtime version for C or C++ beyond the compiler versions.

Java reruns in the current study were executed with DaCapo’s single-iteration, single-thread configuration (`-n 1 -t 1`), together with an explicitly selected GC and matching `-Xms/-Xmx` heap settings. No other flags were used beyond the collector and heap size choices. The primary GC sets in the harness are Serial, Parallel, G1, and ZGC.

The native side used C and C++ ports compiled with O2-level optimization. We explore different levels of optimization within RQ7. Each compiled port was linked with one of three general-purpose allocators: the default system allocator, `jemalloc`, or `mimalloc`.

The performance harness collected wall-clock time, hardware counter data, and resident set size during the same run. Resident set size was sampled every 20 ms from the Linux process, and the maximum observed `VmRSS` was recorded. Hardware-counter collection used Linux’s `perf` and collected data on cycles, instructions, page faults, cache activity, and TLB-related events. Each of these results was collected during every trial. We measured 5 trials per configuration for both Java and native code. We report the medians over successful runs, and no general outlier rule was applied.

We intentionally did not enforce CPU pinning between Java and native runs. Both were left unpinned for fairness. Some thought was given to pinning both runs to isolate concurrent factors. However, modern GCs like ZGC rely on concurrency to achieve competitive performance, whereas GCs that do not rely on concurrency do not. The fact that we keep worker threads to one for both Java and native runs means we keep concurrency out of the main work being done, thus allowing modern GCs to take advantage of their newer strengths rather than unfairly crippling them to make both explicit and managed collection non-concurrent.

3.4 Heap Selection

Heap sizes were chosen relative to a benchmark and GC-specific minimum viable heap. For each Java configuration, we performed a binary search over candidate minimum heap sizes and identified the smallest heap at which the benchmark ran stably. That stable minimum then served as the reference point for subsequent heap sweeps and runs at multiples of that size. We ran a fixed set of multiples: 1.0 $\times$, 1.25 $\times$, 1.5 $\times$, 2.0 $\times$, 3.0 $\times$, 4.0 $\times$, and 6.0 $\times$. The use of multiples of the min heap size was done to follow what was done in Hertz and Berger’s paper and how they performed their space–time tradeoff analysis [1]. The purpose is to determine whether giving Java more memory removes not only small-heap penalties, but also whether increasing heap size is sufficient to close the gap to the native baseline.

3.5 Scope and Limitations

Our methodology is narrower than the original oracle experiment. It does not preserve the exact JVM allocation sites, object identity, runtime services, or compiler pipeline of a managed execution. The results, therefore, should not be interpreted as a purely isolated measure of GC cost. Instead, they map out a modern empirical comparison between managed Java executions and workload-equivalent native baselines.

The use of Claude Code and Claude Opus 4.6 introduces the possibility of implementation bias or valid but incorrectly generated code. We tried to address potential LLM bias or simplification by making validation, manual review, and benchmark-specific equivalence testing the criteria for acceptance, rather than relying solely on model output.

4 Native Benchmark Ports

As mentioned, we are porting a subset of DaCapo 23.11-MR2 Chopin benchmarks to native implementations in order to compare Java GC execution with explicit memory management. The primary ports are written in C, and we aim for workload-equivalent behavior, such that they use

Table 1: **Native ports status for ported DaCapo 23.11-MR2 Chopin benchmarks.**

Benchmark	C port status	C++ port status	Sizes validated
luindex	validated	Lucene-shaped port	small, default
lusearch	validated	Lucene-shaped port	small, default
sunflow	validated	not ported	small, default
xalan	validated	not ported	small, default
h2	validated	not ported	small, default
biojava	validated	Java-shaped pilot port	small, default, large
graphchi	validated	not ported	small, default
zxing	validated	not ported	small, default

the same DaCapo input, the same high-level algorithms, and deterministic outputs. There are also C++ variants, as mentioned before within the methodology section.

c-luindex / cpp-luindex — Lucene text indexing. **Java:** DaCapo’s luindex uses Apache Lucene to index a Wikipedia line-document corpus, exercising tokenization, lowercasing, stop-word filtering, postings construction, field metadata, and segment buffering.

C port: Our C version preserves the main indexing pipeline, including document parsing, tokenization, filtering, term-frequency accumulation, and posting-list construction. The coding agent used native hash tables and dynamic arrays rather than Lucene’s full segment and codec machinery.

C++ port: The C++ version aims to restore more of Lucene’s object structure and implement more of the upstream Lucene library, including analyzer, tokenizer, document, field, reader, and writer abstractions. The port, however, still simplifies low-level codec details.

c-lusearch / cpp-lusearch — Lucene text search **Java:** lusearch opens a Lucene index, loads query files, executes searches through `IndexSearcher`, scores results with BM25-style similarity, traverses postings, maintains top- N hits, and writes outputs.

C port: The C port preserves term lookup, postings traversal, BM25 scoring, top- N selection, and deterministic output generation. However, it uses simpler data structures than Lucene’s full

backend library.

C++ port: The C++ version aims to bring more of Lucene’s library into C++ code, including queries, weights, scorers, similarities, readers, and collectors.

c-sunflow / cpp-sunflow — ray tracing **Java:** sunflow renders a fixed scene, exercising scene setup, ray generation, acceleration-structure traversal, intersection testing, shading, shadows, reflection/refraction, image accumulation, and validation.

C port: The C port aims to preserve the workload with a deterministic Whitted-style ray tracer, including scene construction, traversal, ray casting, shading, and checksum generation. It matches the rendering computation, but it does not match Java’s object allocation or allocation pressure.

C++ port: The C++ version is intended to better preserve DaCapo Sunflow’s original class structure, potentially making the benchmark more closely follow Java’s allocation behavior.

c-xalan — XSLT transformation. **Java:** xalan applies Xalan-J to XML inputs, performing XML parsing, DOM-like tree construction, stylesheet processing, XPath evaluation, result-tree creation, and HTML serialization.

C port: The C port aims to preserve this transformation pipeline, including DOM construction, template matching, XPath-subset evaluation, result-tree construction, and output serialization. However, it does not reproduce every Xalan-J class; instead, it implements only the subset needed for the DaCapo workload while retaining its heavy allocation behavior.

c-h2 — embedded database / TPC-C-style transactions. **Java:** h2 runs an embedded H2 database with a TPC-C-style client, stressing schema creation, SQL execution, indexing, row storage, and the standard transaction mix.

C port: The C port aims to preserve the TPC-C schema, transaction mix, deterministic seed, table state, and indexing behavior. However, it replaces JDBC and SQL parsing with direct native transaction procedures over tables and B-tree-style indexes.

c-graphchi — ALS matrix factorization. **Java:** graphchi runs GraphChi-Java’s alternating-least-squares matrix factorization over ratings data using pre-sharded graph files, vertex intervals, edge blocks, and repeated latent-factor updates.

C port: The C port aims to read the GraphChi-Java shard format directly and preserve the ALS computation, traversal order, sub-interval processing, edge handling, and per-vertex linear algebra.

c-biojava/cpp-biojava — protein-property computation. **Java:** biojava runs BioJava’s `AAProperties` tool on FASTA inputs, cleaning sequences and computing biochemical properties such as molecular weight, extinction coefficients, instability index, GRAVY, isoelectric point, net charge, and amino-acid composition.

C port: The C port aims to preserve the streaming FASTA pipeline and all property calculations, mirroring BioJava’s constants and formatting.

C++ port: The C++ port aims to do the same amount of computation while following more of an object-oriented paradigm.

c-zxing — barcode decoding. **Java:** zxing runs the ZXing barcode reader on image inputs, including loading images, binarization, barcode detection, decoding, error correction, and result recording.

C port: The C port aims to preserve the decoding pipeline for the DaCapo image set, including loading, luminance conversion, binarization, detector execution, decoding, and output validation.

5 Results

Since our study compares workload-equivalent native ports rather than an Oracle-managed JVM, we do not interpret the results as a pure isolation of GC cost. Instead, we use them to evaluate a similar but modern question: how closely tuned Java configurations approach comparable native baselines, how that gap changes with additional heap space, and how much of the remaining difference is associated with memory footprint, dynamic work, microarchitectural behavior, collector choice,

implementation structure, and native compiler optimization.

A useful way to interpret our results is to separate the three effects that are a part of any GC vs EMM comparison. First, GC-managed Java has to deal with certain runtime features that our C and C++ ports do not have. These include collector metadata, barriers, heap management structures, object headers, and runtime services. Second, GC performance depends strongly on heap space, and when the heap is close to the minimum viable size, the collector has to work more frequently to clean up space. Thirdly, the native ports may benefit not only from EMM but also from different implementation structures. These structural differences include simpler data structures, fewer object calls, and more direct control over allocation. Our results can then be viewed as evidence for GC-managed Java versus explicit memory-native baselines.

5.1 RQ1: How do modern Java collectors compare with native baselines?

Table 2 reports the best C configuration and the best Java configuration for each benchmark. Across all eight measured benchmarks, the fastest native baseline is faster than the fastest Java configuration. The size of the gap in wall-clock time varies, from 2.53× on `graphchi` to 26.35× on `lusearch`.

The larger gaps in `h2`, `biojava`, and `lusearch` may not be as clean measures of GC overhead alone, as the agentic coding system made substantial simplifications. For `h2`, the native implementation preserves the transaction mix but omits substantial parts of the Java H2 stack, including JDBC and the full SQL execution path. For `biojava`, the C implementation is output-valid but streams through reusable buffers, making it much less allocation-heavy than the Java workload. For `lusearch`, the compact C implementation removes much of Lucene’s object structure. These benchmarks, therefore, show the range of possible Java-native differences under AI-assisted porting. These results more directly show how large the Java/native separation can become when the native port removes more of the framework’s structure or object-model overhead.

`graphchi` and `luindex` serve as good anchor cases for the paper due to their high structural similarity and validity. In `graphchi`, the best native configuration runs in 9.68 s, while the best

Java configuration runs in 24.52 s, for a Java/native ratio of 2.53 $\times$. In `luindex`, the best native configuration runs in 11.26 s, while the best Java configuration runs in 38.22 s, for a 3.40 $\times$ gap. These cases matter because they show that even in native ports that preserve more of the original workload structure, a Java run with more heap space still does not reach native performance, thus showing that there is still a noticeable gap in performance between GC and EMM that potentially exists and is not just the result of extreme cases.

Figure 1 for `luindex` illustrates the pattern that the size of the gap depends strongly on workload structure and validation. By examining `luindex`, we see that compact C is fastest, the C++ probes occupy an intermediate regime, and the best Java GC configurations remain slowest. We use `luindex` as a visual here because it combines strong validation with a clear structural gradient from compact C to more Java-like C++ to managed Java. The C++ ports will be discussed further in RQ6.

For the GC versus EMM, the results suggest that the mechanisms that make Java managed are not fully offset by heap and collector selection for these benchmarks. EMM-style native execution can avoid much of this runtime overhead and, as shown in our native port, can use more direct representations. In examining `graphchi` and `luindex` shows that the advantage of native execution is still maintained.

Answer to RQ1

Across all eight measured workloads, the fastest native baseline outperforms the fastest Java configuration. The smallest gaps appear in the anchor cases `graphchi` (2.53 $\times$) and `luindex` (3.40 $\times$), while the largest gaps appear in more structurally conditional benchmarks such as `biojava` (10.31 $\times$), `h2` (12.71 $\times$), and especially `lusearch` (26.35 $\times$). Taken together, the table shows that tuned Java does not achieve tuned-native performance anywhere in the suite, though the size of the gap depends strongly on workload and implementation structure. Thus, there is no single quantitative result that fully describes the gap.

Benchmark	Fastest native	Native (s)	Fastest Java	Collector	Heap (MB)	Java (s)	Java/native
biojava	c-jemalloc	2.59	java-serial-Xmx190m	Serial	190	26.72	10.31×
graphchi	c-jemalloc	9.68	java-parallel-Xmx1194m	Parallel	1194	24.52	2.53×
h2	c-mimalloc	10.45	java-parallel-Xmx1614m	Parallel	1614	132.92	12.71×
luindex	c-glibc	11.26	java-serial-Xmx150m	Serial	150	38.22	3.40×
lusearch	c-mimalloc	3.14	java-g1-Xmx114m	G1	114	82.86	26.35×
sunflow	c-mimalloc	20.24	java-g1-Xmx90m	G1	90	127.28	6.29×
xalan	c-mimalloc	6.60	java-parallel-Xmx30m	Parallel	30	29.74	4.50×
zxing	c-jemalloc	5.70	java-parallel-Xmx96m	Parallel	96	40.05	7.03×

Table 2: **Native remains faster than Java in every benchmark, even after Java is tuned over collector and heap size.** The table lists the fastest observed native baseline and the fastest observed Java configuration for each workload, together with the corresponding Java/native runtime gap. An anomaly seen is that **zxing** results are deceptive, as the execution times found are most likely an outlier that is later discussed in RQ2.

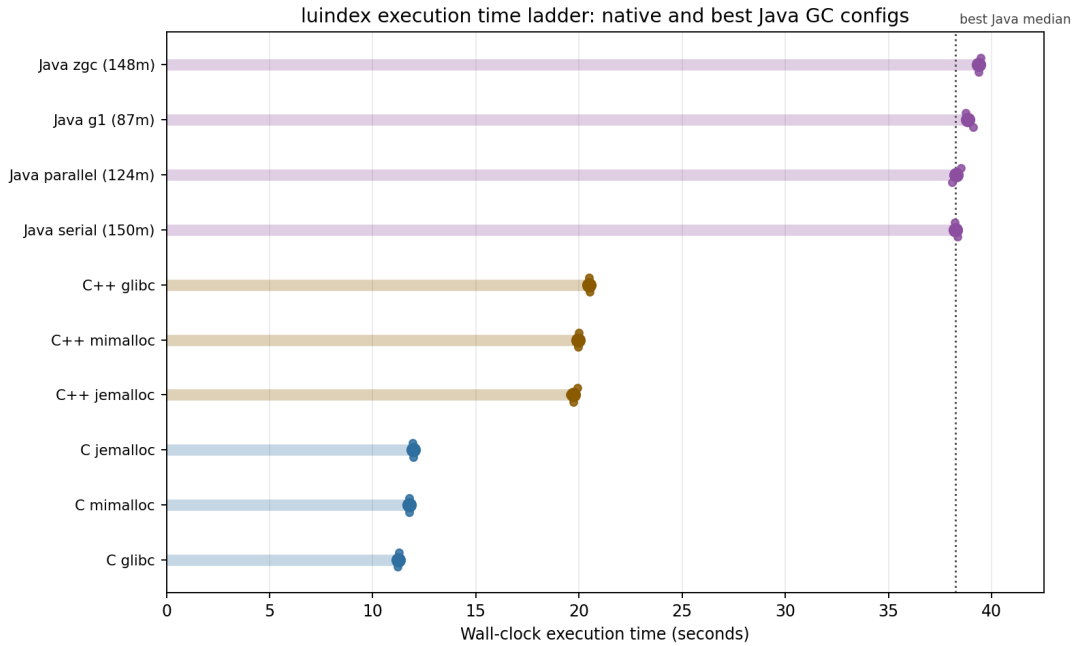


Figure 1: **Execution-time ladder for luindex across compact C baselines, C++ probes, and the best Java GC configurations.** Compact C is fastest, the more structurally faithful C++ variants occupy an intermediate regime, and the best Java GC configurations remain slowest.

5.2 RQ2: Does giving Java more heap space close the gap?

Figure 2 and Table 3 summarize the heap-sweep results. We examine Java GCs across multiple heap sizes relative to the minimum heap required to run the program. The figures show that increasing heap space often removes small-heap penalties, but many curves plateau well above the best native result.

The benchmarks fall into three broad groups. The first group shows a strong small-heap cliff followed by a plateau, most clearly in `graphchi`, `h2`, `sunflow`, and `xalan`. In these cases, the smallest heaps impose a large runtime penalty, and most of the improvement occurs once the heap becomes large enough. The second group shows moderate improvement, followed by a plateau, as in `lusearch` and, to a lesser extent, `biojava`. The third group is mostly flat, with `zxing` showing the clearest example and `luindex` showing only mild improvement relative to the native baseline.

The `zxing` results should be interpreted cautiously. In the normalized heap-sweep figure, most runs appear unusually close to the native baseline, while the `jemalloc` configuration stands out as an outlier. We therefore treat `zxing` as an anomalous or methodologically weaker case rather than as a counterexample to the broader pattern. The results are most likely due to faulty porting behavior. The fast execution time of `jemalloc` should also be observed cautiously. As in other results, we see that changing the allocator has no significant effect on the benchmarks’ execution time. So this anomalous behavior should be treated as just that, not as some unique discovery.

In `graphchi`, performance improves by 81.8% between the best small-heap Java run and the best observed Java configuration, yet Java still remains $2.53\times$ slower than the best native baseline. In `luindex`, larger heaps reduce memory pressure with only modest improvements. The best Java configuration improves by 8.1% relative to the smallest tested heap, and the plateau still remains $3.40\times$ above the best native runtime.

This heap sweep experiment is the closest analog to Hertz and Berger’s original space–time result. When Java is given very little heap, GC has less room to defer reclamation and must spend more time managing memory. As heap size increases, the collector can often run less frequently or with less pressure, which explains the steep early improvements in benchmarks such as `graphchi`,

Benchmark	Native (s)	Smallest heap	Best Java at small heap (s)	Best Java config	Best heap	Best Java (s)	Largest heap	Best Java at largest heap (s)	Small→best improvement	Best Java/native
biojava	2.59	119	28.55	java-serial-Xmx190m	190	26.72	1200	33.14	6.4%	10.31×
graphchi	9.68	173	134.96	java-parallel-Xmx1194m	1194	24.52	1194	24.52	81.8%	2.53×
h2	10.45	710	177.77	java-parallel-Xmx1614m	1614	132.92	6636	201.76	25.2%	12.71×
luindex	11.26	25	41.58	java-serial-Xmx150m	150	38.22	222	39.58	8.1%	3.40×
lusearch	3.14	15	155.95	java-g1-Xmx114m	114	82.86	150	87.81	46.9%	26.35×
sunflow	20.24	15	413.12	java-g1-Xmx90m	90	127.28	144	131.75	69.2%	6.29×
xalan	6.60	5	85.10	java-parallel-Xmx30m	30	29.74	54	34.76	65.0%	4.50×
zxing	5.70	48	40.47	java-parallel-Xmx96m	96	40.05	288	40.05	1.0%	7.03×

Table 3: **Larger heaps often remove severe small-heap penalties, but the Java curves still plateau above the native baseline.** The table summarizes, for each benchmark, the smallest tested heap, the best observed Java configuration, the largest tested heap, the improvement from small heap to best Java result, and the remaining Java/native gap.

sunflow, and xalan. However, once the curves plateau, the remaining gap is no longer explained only by too little heap. At that point, the gap likely reflects the broader cost stack of managed execution. In contrast, the native baselines reclaim memory explicitly and often execute with more compact data structures, so they do not pay the same heap management costs.

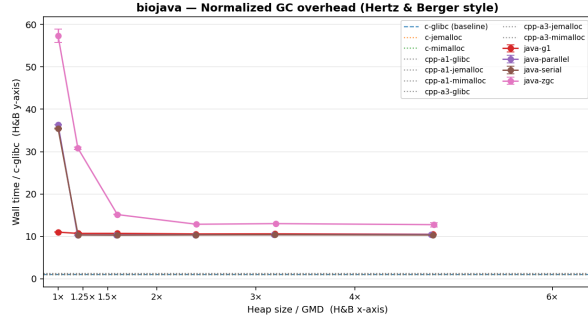
Answer to RQ2.

Additional heap space helps Java substantially near the minimum viable heap, but it does not close the gap to the native baseline. The modern space–time tradeoff still appears, but in this study, the Java curves plateau before reaching native performance.

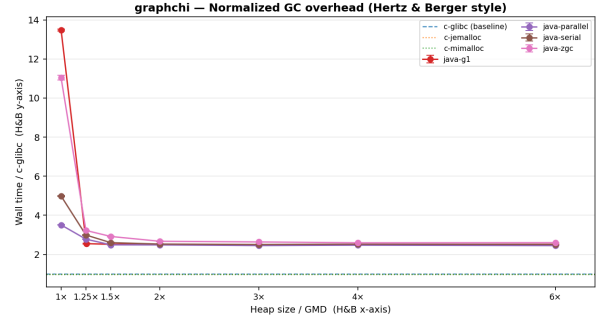
5.3 RQ3: How do Java and native execution differ in memory footprint?

Table 4 reports peak resident-set size and dynamic instruction counts for the best native and best Java configurations, while Figure 3 shows how the RSS grows over time for the `luindex` benchmark. The suite-level pattern is that Java usually has a larger RSS than the corresponding native baseline.

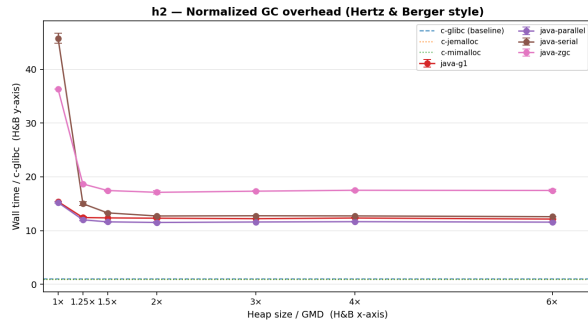
This result is not surprising, as Java runs include the managed runtime, collector metadata, heap-management structures, and the benchmark object graph itself, as well as many nested allocations that all incur additional memory cost. Native ports do not require the same runtime machinery, so the expected footprint is usually lower.



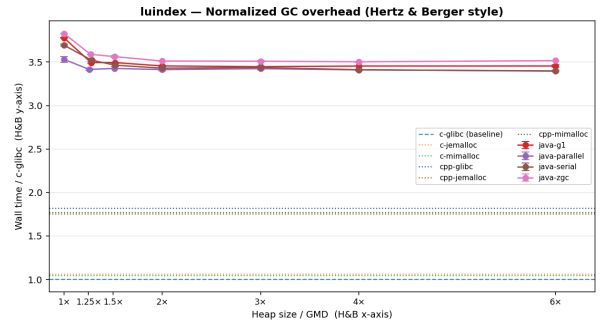
(a) biojava



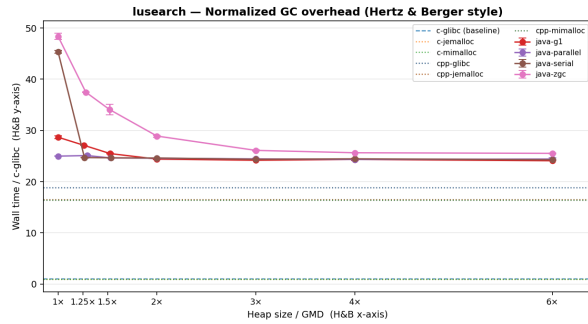
(b) graphchi



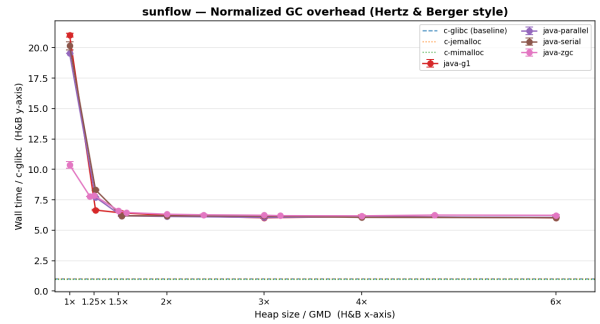
(c) h2



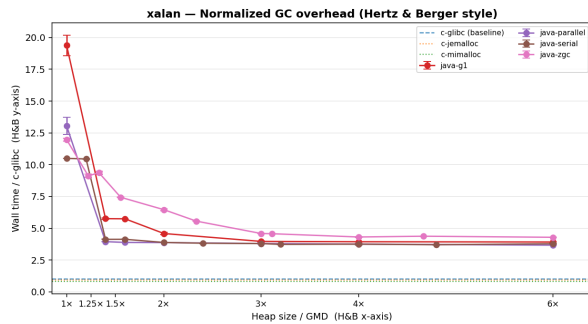
(d) luindex



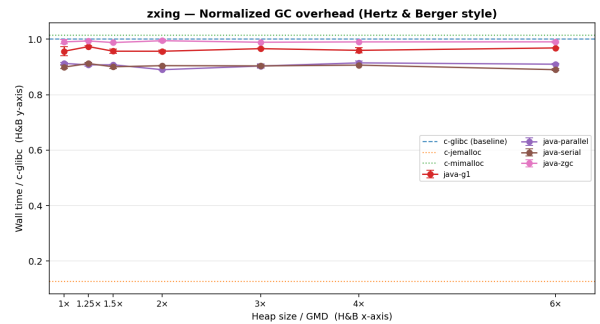
(e) lusearch



(f) sunflow



(g) xalan



(h) xzing

Figure 2: **Heap-sweep wall-clock time normalized to the c-glibc baseline for all benchmarks.** Across most workloads, the key visual pattern is a steep early drop followed by plateauing above native performance. Additional heap space often removes severe small-heap performance penalties, but it typically does not bring Java down to the native baseline.

Once again, the benchmarks fall into broad groups. Java shows substantially larger RSS in `graphchi`, `sunflow`, `xalan`, `zxing`, and `biojava`, and a moderate RSS increase in `luindex` and `lusearch`, as well as one oddity in `h2`, where the best Java configuration has a slightly lower peak RSS than the best native configuration. This oddity could be due to the implementation structure the coding agent produced, which could be less memory-efficient than the original code. As well, the port does not use the direct JDBC and SQL parsing path and instead uses B-tree-style indexes, which is likely to lead to less efficient memory usage. This exception matters because it shows that the native baseline is not automatically smaller and that benchmark porting may not be perfect. It also reinforces the idea that the Java and native implementations differ substantially in system structure.

In `graphchi`, Java uses 908.53 MiB compared with 96.25 MiB for the best native run, or $9.44\times$ the native RSS. In `luindex`, Java uses 315.93 MiB compared with 165.19 MiB for C, or $1.91\times$ the native RSS. Figure 3 further shows that over time, Java not only has a higher peak, but sustains a larger footprint for much of the execution, thus suggesting poorer memory management, as it holds more memory for longer than needed.

Figure 3 also shows an RSS view that includes the C++ probes. Compact C occupies the smallest footprint, C++ moves upward, and the Java configurations remain the highest for most of the execution. The results show that even with more Java-like structure in C++, the increased structure may raise memory costs, but there is not only a memory gap but also an execution gap.

The RSS results show the space side of the GC vs EMM tradeoff. GC is designed to reclaim memory automatically, and as shown, the added headroom lets the collector delay work, increasing its memory footprint for much of its execution. The native EMM-style ports can release or reuse memory directly, avoiding the added overhead. Thus showing the optimal advantages that EMM has over Java benchmarks. However, RSS alone can not prove this optimality and examines only the GC cost.

Benchmark	Fastest native	Native (s)	Native RSS (MiB)	Native instr. (B)	Fastest Java	Java (s)	Java RSS (MiB)	Java instr. (B)	Java/native RSS	Java/native instr.
biojava	c-jemalloc	2.59	9.26	7.89	java-serial-Xmx190m	26.72	249.42	109.76	26.92×	13.91×
graphchi	c-jemalloc	9.68	96.25	43.48	java-parallel-Xmx1194m	24.52	908.53	202.92	9.44×	4.67×
h2	c-mimalloc	10.45	1935.08	20.19	java-parallel-Xmx1614m	132.92	1807.49	393.85	0.93×	19.51×
luindex	c-glibc	11.26	165.19	19.96	java-serial-Xmx150m	38.22	315.93	179.07	1.91×	8.97×
lusearch	c-mimalloc	3.14	71.89	8.18	java-g1-Xmx114m	82.86	248.11	276.31	3.45×	33.79×
sunflow	c-mimalloc	20.24	9.27	70.81	java-g1-Xmx90m	127.28	152.61	395.97	16.47×	5.59×
xalan	c-mimalloc	6.60	9.26	24.56	java-parallel-Xmx30m	29.74	146.93	73.39	15.86×	2.99×
zxing	c-jemalloc	5.70	17.68	17.73	java-parallel-Xmx96m	40.05	199.36	175.37	11.28×	9.89×

Table 4: **Java usually executes with both a larger resident footprint and more dynamic work than the corresponding native baseline.** The table compares the best native and best Java configurations for each benchmark using wall-clock time, peak RSS, retired instructions, and Java/native ratios for RSS and instruction count.

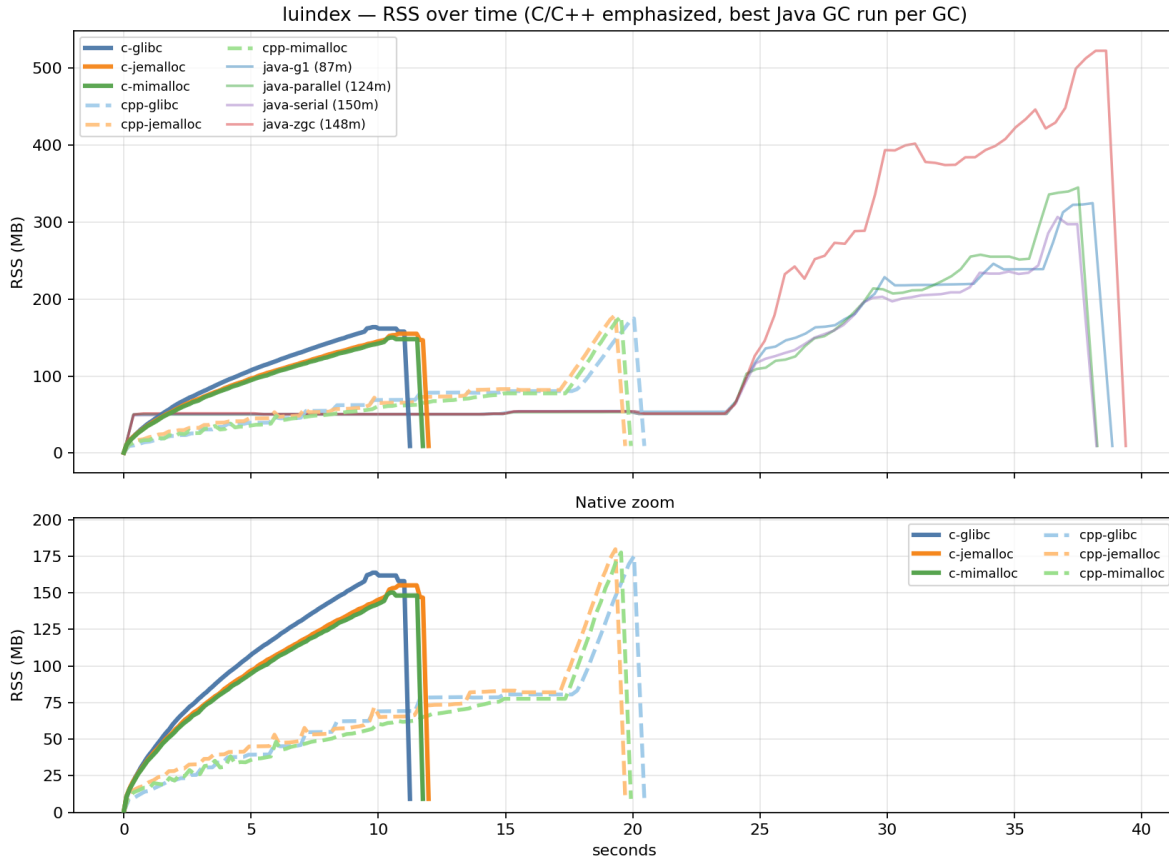


Figure 3: **Detailed RSS-over-time view for luindex, emphasizing compact C baselines, C++ probes, and the best Java run for each collector family.** The C++ ports occupy an intermediate footprint regime between compact C and Java, while the Java runs sustain the largest resident footprint for most of the execution.

Answer to RQ3. Java generally uses more resident memory than the corresponding native baseline, often by a wide margin, and the difference is usually sustained over much of the execution rather than appearing only at the peak. At the same time, cases such as `h2` show that RSS alone is insufficient for analyzing true GC cost, since the implementation structure also contributes to the observed footprint gap. Different language paradigms also contribute to the large differences in RSS, not just how they manage their own memory. However, this is part of the trade-off of having a more secure memory system in Java than in an EMM-oriented language such as C/C++.

5.4 RQ4: What microarchitectural differences accompany the runtime gap?

Table 5 reports selected microarchitectural results for the best native and best Java configurations. There is not one single metric that paints the whole picture, but examining them together helps. The main difference is that Java does more total work and often exhibits worse locality of reference.

The benchmarks divide naturally into more work-dominated, locality-dominated, or mixed cases. `Luindex` is one case that could be viewed as more work-dominated. Java executes $8.97\times$ as many instructions as the best native run, even though its IPC (Instructions Per Cycle) is actually higher (1.36 versus 0.79) and both L1D and dTLB MPKI are lower than in native. Our results suggest that the runtime gap in `luindex` is not explained only by poorer cache or TLB behavior, but also by greater total executed work for the Java program.

By contrast, `graphchi` and `xalan` are stronger locality-dominated or mixed cases. In `graphchi`, Java has both lower IPC and worse locality: IPC falls from 1.98 to 1.16, L1D MPKI rises from 0.70 to 3.64, and dTLB MPKI rises from 0.19 to 1.49. In `xalan`, Java again combines lower throughput with much worse miss intensity, with L1D MPKI increasing from 4.79 to 16.81 and dTLB MPKI from 1.83 to 14.61. These cases suggest that the Java slowdown is not only about doing more work, but also about doing that work with worse locality. The remaining benchmarks continue to show mixed results. Altogether, the results show how Java’s runtime disadvantage in both dimensions

Benchmark	Native IPC	Java IPC	Native L1D MPKI	Java L1D MPKI	Native dTLB MPKI	Java dTLB MPKI
biojava	1.35	1.50	0.28	2.82	0.14	1.75
graphchi	1.98	1.16	0.70	3.64	0.19	1.49
h2	0.87	0.93	6.13	11.47	3.57	12.54
luindex	0.79	1.36	8.82	5.47	8.03	5.41
lusearch	1.19	1.25	4.96	5.09	4.66	2.55
sunflow	1.53	1.31	1.22	3.37	0.72	1.61
xalan	1.64	0.70	4.79	16.81	1.83	14.61
zxing	1.77	1.43	0.44	4.58	0.09	3.91

Table 5: **IPC, L1D miss intensity, and dTLB miss intensity for the best native and best Java configurations.** The runtime gap is often accompanied by worse locality, more total work, or both on the Java side. MPKI denotes misses per thousand instructions.

could be impacting its ability to achieve more efficient memory behavior.

Answer to RQ4.

Java is usually slower because it performs more total work, and in several workloads, it also performs that work with worse locality. Some benchmarks, such as `luindex`, are primarily work-dominated, while others, such as `graphchi` and `xalan`, show both greater work and clearly worse memory-system behavior. The suite-level pattern is therefore mixed but somewhat consistent. The Java/runtime gap is supported by greater dynamic work, worse locality, or both.

5.5 RQ5: How much does collector choice matter?

Table 6 reports the best observed runtime for each GC on each benchmark. No single collector dominates across the full suite. Serial is best for `biojava` and `luindex`; Parallel is best for `graphchi`, `h2`, `xalan`, and `zxing`; and G1 is best for `lusearch` and `sunflow`. The table shows that there is no one-size-fits-all GC. However, as noted in Figure 2, many of the GCs end up plateauing at similar performance, except more often than not, ZGC.

The main finding is that it does not matter which collector “wins” for each benchmark as much as one might expect. Collector choice changes the best Java run by much less than the difference between Java and native runs. In several benchmarks, the best-case spread among collectors is

Benchmark	Serial (s)	Parallel (s)	G1 (s)	ZGC (s)	Spread
biojava	26.72	26.84	27.26	33.14	24.03%
graphchi	25.08	24.52	24.80	25.95	5.85%
h2	145.40	132.92	140.25	197.76	48.78%
luindex	38.22	38.26	38.80	39.43	3.16%
lusearch	83.56	83.71	82.86	87.81	5.98%
sunflow	127.53	128.10	127.28	130.82	2.78%
xalan	30.02	29.74	31.74	34.76	16.85%
zxing	40.05	40.05	43.04	44.46	11.01%

Table 6: **Collector choice changes the best Java result, but usually by much less than the overall Java/native gap.** The table reports the best observed runtime for each collector family on each benchmark, and the spread between the fastest and slowest collector best cases.

small: 3.16% in `luindex`, 5.85% in `graphchi`, 5.98% in `lusearch`, and 2.78% in `sunflow`. The choice of Java collector, therefore, does not really affect the headline result that the native runs remain faster than the Java configurations.

Answer to RQ5.

Collector choice matters for fair Java evaluation, but it is not the main driver of the comparison between Java and native. Different collectors win on different workloads by a small margin when the heap size increases, yet the spread among collectors is usually much smaller than the overall Java/native gap.

5.6 RQ6: What do the C++ ports reveal about implementation structure?

Table 7 and Figure 4 summarize the C++ results in relation to both the Java runs and the C runs. These ports show changes in implementation structure, allowing us to see not only how an LLM can translate more Java-like structure, but also whether the gap remains in another EMM language. The C++ ports also help to examine whether restoring more object-oriented structure moves the baselines closer to Java.

In `lusearch`, the compact C version runs in 3.14 s, while the C++ version runs in 56.50 s, a

17.97 $\times$ slowdown relative to C. Java still runs slower at 82.86 s, but the distance from C++ to Java is much smaller than the distance from compact C to Java. This suggests that a large part of the original gap in `lusearch` is structural. Once the port restores more Lucene-like object structure, much of the cost removed by compact C reappears. However, this slowdown in performance could also stem from the extreme efficiency of the Lucene Java library, which the C++ port's replication may not fully capture.

`luindex` shows the same pattern, but to a lesser extent. The best C version runs in 11.26 s, while the best C++ version runs in 19.74 s, for a 1.75 $\times$ slowdown relative to compact C. Java remains slower at 38.22 s. It seems that restoring a more Java-like structure may yield a more accurate comparison to Java; however, the gap between native and EMM remains. Thus, the gap that has been found cannot be explained by compactness alone.

`biojava` provides a different case. The base C runs in 2.59 s, and C++ runs in 3.22 s, a much smaller 1.24 $\times$ spread than in the Lucene-based workloads. Java remains far slower at 26.72 s. This shows that simply introducing a C++ port version does not automatically restore Java-like behavior. A proper port depends more on whether it reproduces the relevant allocation patterns and object graphs of the original workload than on language alone.

The C++ ports support the idea that the gap found is not specific to C ports. In fact, they show the advantage that native execution can have and that EMM-style manual memory management can still preserve.

Benchmark	Best C (s)	Best C++ (s)	Fastest impl.	Slowest impl.	Spread
biojava	2.59	3.22	c	cpp-biojava	1.24×
luindex	11.26	19.74	c	cpp	1.75×
lusearch	3.14	56.50	c	cpp	17.97×

Table 7: **Restoring more Java-like structure in C++ recovers part of the gap between compact C and Java, especially for lusearch.** The table summarizes benchmarks across multiple native implementations and shows how the runtime changes when moving from compact C to more structurally faithful C++ probes.

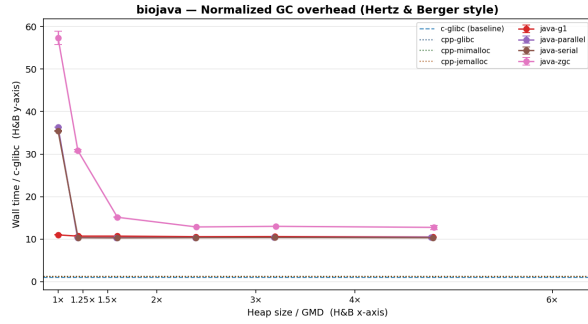
Answer to RQ6.

The C++ probes show that implementation structure matters. In `lusearch` and `luindex`, restoring more object-oriented structure narrows the Java/native gap by recovering a substantial portion of the cost removed by compact C. In `biojava`, by contrast, the C++ probe changes little, showing that language choice is less important than whether the port reproduces the relevant workload shape and allocation behavior. Thus showing the weakness and potential pitfalls that porting via LLMs can introduce. Part of the native advantage is therefore structural, but a meaningful gap still remains even after a more Java-like structure is restored.

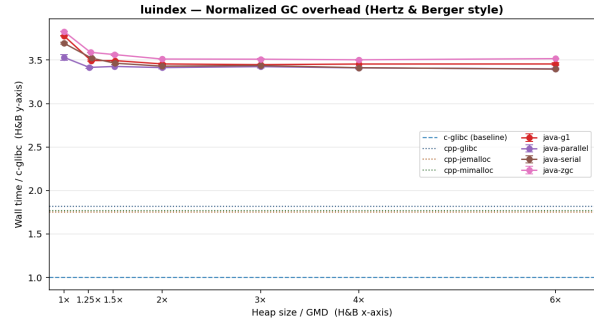
5.7 RQ7: How sensitive is the observed Java/native gap to optimization of the native baseline?

To test whether the observed Java/native performance gap depends strongly on how the native baseline is compiled, we performed compiler optimization sweeps across the glibc native builds and compared them with the best-observed Java configuration for each GC. We then examine whether the GC vs. EMM gap is dependent on native compiler efficiency.

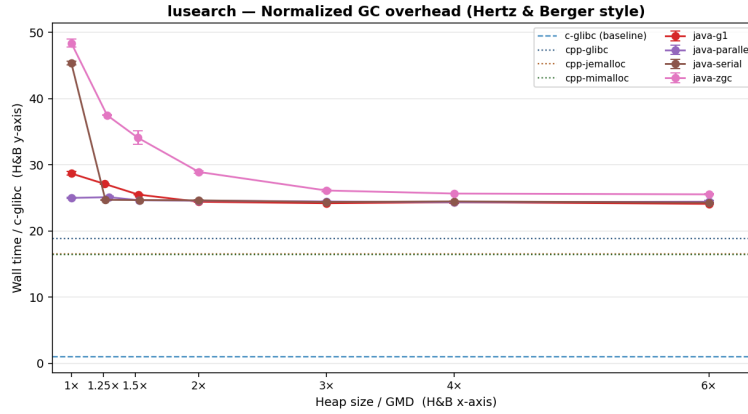
Figure 5 shows that native compiler optimization affects the absolute baseline, but not as dramatically as expected. In `graphchi`, the largest improvement occurs when moving away from `O0`, after which the native runtime remains well below the best Java configurations. In `luindex`, the same pattern appears more smoothly. Optimization lowers the native baseline, but the best



(a) biojava

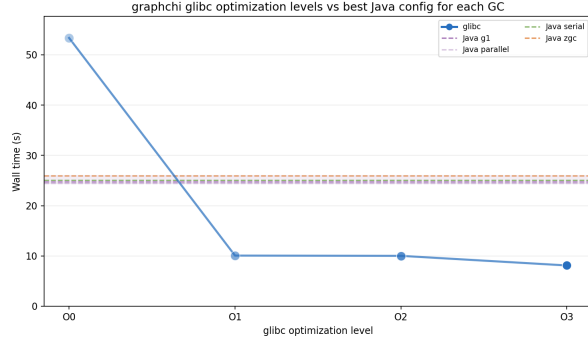


(b) luindex

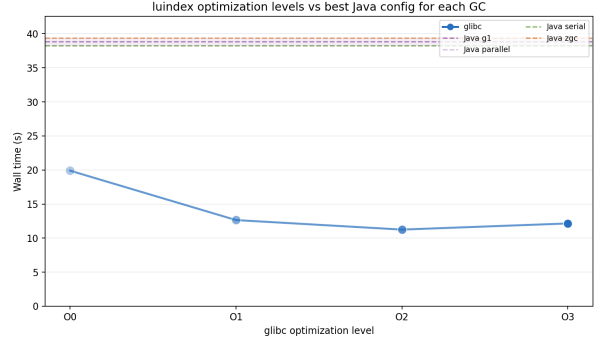


(c) lusearch

Figure 4: **Heap-sweep wall-clock time normalized to the c-glibc baseline for the benchmarks that include C++ probe implementations.** The results show that additional heap space often reduces the worst Java slowdowns near the minimum viable heap, but the curves typically plateau above the native baselines. They also show that implementation structure matters. In luindex and especially lusearch, the C++ probes narrow the gap between compact C and Java, suggesting that part of the observed Java/native difference is explained by port structure rather than by memory management alone.



(a) graphchi



(b) luindex

Figure 5: **Sensitivity of the native glibc baseline to compiler optimization level for graphchi and luindex.** These checks show that native optimization changes the absolute baseline, but does not overturn the overall ordering between optimized native and the best Java runs. Those Java runs are being executed to completion as quickly as possible.

Java collector results still remain far above the optimized native builds. However, both examined benchmarks show little improvement beyond 00. Thus, optimization can widen the measured gap relative to 00, but it does not provide any large new finding about the native and Java results.

Answer to RQ7.

Native compiler optimization affects the absolute runtime baseline and can widen the measured Java/native gap, especially when comparing 00 against 01/02 builds. However, not enough to say that the gap either worsens or improves generally by increasing the level of optimization.

6 Discussion

This paper does not suggest that garbage collection is good or bad. As with any programming language, and more specifically, memory management strategy, there are benefits and costs. Instead, as we have mentioned, we are reexamining the GC vs. EMM result from Hertz and Berger’s paper. We agree that heap space can substantially improve Java performance. However, under the methodology employed here, Java tends to plateau in performance before reaching our native baselines for both C and C++ ports, except for the noted exceptions. Our findings suggest that Java and GC, even when given ample resources, may not fully match the performance of C/C++ and

EMM, providing insight that the original study may have been too conservative in its attempt to replicate EMM-like behavior.

A strong example showing our findings and our claim can be found in `luindex` and `graphchi`. Benchmarks that we have noted before as having a more similar structure and allocation workload to their Java counterparts. In `graphchi`, Java improves by 81.8% from the best small-heap run to the best observed Java configuration, yet still remains $2.53\times$ slower than the best native baseline. In `luindex`, Java improves only modestly with additional heap, and the best Java result still remains $3.40\times$ above native. This reiterates that even with heap growth, a gap still exists, and increased memory may not be able to close it. As well, by examining RSS overtime, Java and its GC have to hold onto much more memory than the ported C benchmarks had to hold onto. Thus even without having small heap sizes that results in more aggressive collector behavior and therefore longer pause times, the large memory footprint that gets created does not help close the gap to the native baselines.

Our results remain consistent with current work showing that GC costs, despite modern advancements, remain high [3]. Even after increasing heap size and changing collectors, the best Java runs consistently remained above the native baselines. We also examined more than just heap-size comparisons and showed that the GC vs. EMM story is more than any single result [9]. The gap we found can come from multiple factors, such as workload structure in the generated ports, lower RSS in the native baselines than in Java runs, or the fact that Java programs tend to execute more total work. As noted, there may be multiple factors beyond differences in heap size that contribute to the observed gap between automatically managed memory and EMM. Especially that need to be examined as our methodology is not just a pure isolation of GC cost alone and introduces programmatic differences. We do have some differing results from parts of the literature where Huang et al. [2] show that GC can sometimes improve locality. While our results do not disprove this, our results more strongly show that locality is not uniformly favorable to managed memory for the workloads we examined. Our results align more closely with those of Carpen-Amarie et al. [6], who show that modern GC costs are more workload-dependent than a single universal cache

story can explain. Their work supports our findings from our observations in the microarchitectural results that there is no single universal cache story across benchmarks and GCs.

No current general-purpose allocator dominates the native side. As shown in Figure 1 and Figure 2c, the overall runtimes across ports do not differ much from allocator to allocator. Despite advancements in such allocators, they do not seem to offer as much performance benefit in this study as might be expected. The results may be due to the simplicity of the ports these allocators work with, which may not expose their strengths.

Our results fit the GC vs EMM debate by showing that the difference is not simply one system being better than the other. The heap-sweep graphs show the classic space–time tradeoff. Giving Java more memory often reduces GC pressure and improves performance, especially near the minimum viable heap. However, the plateaus show that heap growth alone does not remove the remaining managed runtime cost. The RSS, instruction-count, and microarchitectural results explain why Java can often carry a larger resident footprint, perform more dynamic work, or interact less favorably with the memory hierarchy. The C++ probes further show that some of the measured gap is due to the implementation structure rather than GC alone.

Taken together, these results sharpen rather than settle the GC vs. EMM debate. They support the continued relevance of the original space–time tradeoff, but suggest that in this modern workload-equivalent comparison, giving Java more heap is not sufficient by itself to bring managed execution down to the best native baselines.

6.1 Abnormal and Methodologically Conditional Cases

Not all benchmarks should carry equal weight. `graphchi` and `luindex` remain the stronger anchor cases because their ports preserve more of the original workload structure, and they also produce the smallest gaps in the suite. The two benchmarks mentioned allow us to be a bit more conservative about the quantitative results than the 26.35× gap suggested by `lusearch`. `biojava`, `h2`, and `lusearch` likely include more structural simplification in the native baseline, so their large performance gaps should be taken as more consistent with the general trend that a gap does exist,

but not as a single quantitative number that can be applied across all benchmarks.

These cases are still useful. H2 shows that RSS alone is insufficient to explain the gap, since Java is much slower on that benchmark despite slightly lower peak RSS. `biojava` and `lusearch` show how much the gap can widen when the native implementation becomes much more compact than the original Java structure. The C++ probes partially recover the missing structure, especially in the `lucene` benchmarks, which supports their inclusion.

6.2 Limitations

The main limitation of this study is that it does not preserve the exact Java program while varying only the memory-management policy. The oracle manager remains the major advantage of the original Hertz and Berger oracle methodology, and it is why the present results should not be read as a pure isolation of GC cost. We also cannot provide a concrete performance gap number as Hertz and Berger did. Another limitation is that the native ports were generated through an LLM-assisted workflow, which may introduce systematic design choices that differ from those of a human developer. These limitations weaken the claims, but they do not invalidate the results.

7 Conclusion

This paper revisited the GC-versus-EMM question in a modern setting using eight DaCapo 23.11 workloads, modern Java collectors, modern general-purpose allocators, and native ports, to address the ongoing debate over GC vs. EMM. Across the suite, the fastest native configuration outperformed the fastest Java configuration. Increasing heap size often removed severe small-heap performance drops and improved Java performance. However, in no benchmark did it bring Java down to the level of the best native runtime. Taken together, these results suggest a narrower but important modern conclusion: under the methodology used here, despite newer collectors and greater resources for Java, a performance gap in managed memory exists that previous research did not uncover in the way we did. As we have stated, Java, and by extension GC, did not achieve the same level of

performance, ranging from $2.53\times$ to $26.35\times$ relative to the native baseline execution time.

For future work, we would like to see, with more time and rigorous testing, how many additional ports could be produced using the same methodology. We hope to see whether the pattern continues not only across the DaCapo benchmark suite but also across other modern benchmarks. We would also want to examine more C++ ports rather than C ports, as the former can offer ports without oversimplifying some of the structural patterns present in Java. We would also be interested in investigating Rust ports of Java programs to see whether the gap we examined occurs across multiple EMM languages, thereby lending more credence to the results of this paper.

8 Artifacts/Repo

Ownership may also be transferred to the PLASMA Lab once this school email gets deleted GitHub repository of code used to generate results and the location of C/C++ ports: https://github.com/llasic7558/revisit_quantifying_gc_benchmarks GitHub repository hoisting data and a copy of the paper: https://github.com/llasic7558/revisit_quantifying_gc_benchmarks_paper

9 References

- [1] Matthew Hertz and Emery D. Berger. 2005. Quantifying the performance of garbage collection vs. explicit memory management. In *Proc. OOPSLA '05*, 313–326. <https://doi.org/10.1145/1094811.1094836>
- [2] Xianglong Huang, Stephen M. Blackburn, Kathryn S. McKinley, J. Eliot B. Moss, Zhenlin Wang, and Perry Cheng. 2004. The garbage collection advantage: improving program locality. In *Proc. OOPSLA '04*, 69–80. <https://doi.org/10.1145/1028976.1028983>
- [3] Zixian Cai, Stephen M. Blackburn, Michael D. Bond, and Martin Maas. 2022. Distilling the real cost of production garbage collectors. In *Proc. ISPASS '22*, 46–57. <https://doi.org/10.1109/ISPASS55109.2022.00005>
- [4] Stephen M. Blackburn et al. 2006. The DaCapo benchmarks: Java benchmarking development and analysis. In *Proc. OOPSLA '06*, 169–190. <https://doi.org/10.1145/1167473.1167488>
- [5] Andrew Borg, Andy J. Wellings, Christopher D. Gill, and Ron K. Cytron. 2006. Real-time memory management: life and times. *Proc. ECRTS 2006*, 237–250. <https://doi.org/10.1109/ECRTS.2006.21>
- [6] Maria Carpen-Amarie, Georgios Vavouliotis, Konstantinos Tovletoglou, Boris Grot, and Rene Mueller. 2023. Concurrent GCs and modern Java workloads: a cache perspective. In *Proc. ISMM '23*. <https://doi.org/10.1145/3591195.3595269>
- [7] Matthew Hertz, Stephen M. Blackburn, J. Eliot B. Moss, Kathryn S. McKinley, and Darko Stefanović. 2006. Generating object lifetime traces with Merlin. *ACM TOPLAS* 28, 3 (2006), 476–516. <https://doi.org/10.1145/1133651.1133654>
- [8] Wenyu Zhao and Stephen M. Blackburn. 2020. Deconstructing the Garbage-First collector. In *Proc. VEE '20*, 15–29. <https://doi.org/10.1145/3381052.3381320>

- [9] Kunal Sareen and Stephen M. Blackburn. 2022. Better understanding the costs and benefits of automatic memory management. In *Proc. MPLR '22*, 29–44. <https://doi.org/10.1145/3546918.3546926>
- [10] Oracle. 2018. JEP 318: Epsilon: a no-op garbage collector (experimental). <https://openjdk.org/jeps/318>
- [11] Jason Evans. 2006. A scalable concurrent malloc(3) implementation for FreeBSD. In *Proc. BSDCan*.
- [12] Sanjay Ghemawat and Paul Menage. 2009. TCMalloc: thread-caching malloc. Google Research.
- [13] Daan Leijen, Benjamin Zorn, and Leonardo de Moura. 2019. Mimalloc: free list sharding in action. In *Proc. APLAS '19*. https://doi.org/10.1007/978-3-030-34175-6_1
- [14] Microsoft. 2021. mi-malloc: performance benchmarks. <https://microsoft.github.io/mimalloc/bench.html>
- [15] Hyperledger Besu. 2023. malloc vs. tcmalloc vs. jemalloc vs. mimalloc memory utilization study.
- [16] Theo Ezell. 2025. G1, ZGC, and Shenandoah: OpenJDK's garbage collectors for very large heaps. IBM TechXchange Community.
- [17] Codrut Stancu, Christian Wimmer, Stefan Brunthaler, Per Larsen, and Michael Franz. 2015. Safe and efficient hybrid memory management for Java. In *Proc. ISMM '15*, 81–92. <https://doi.org/10.1145/2754169.2754185>
- [18] Aleksey Shipilev. 2020. JEP 379: Shenandoah: a low-pause-time garbage collector (production). OpenJDK. <https://openjdk.org/jeps/379>
- [19] Stefan Karlsson. 2023. JEP 439: Generational ZGC. OpenJDK. <https://openjdk.org/jeps/439>

- [20] Nathan P. Ricci, Samuel Z. Guyer, and J. Eliot B. Moss. 2013. Elephant tracks: portable production of complete and precise GC traces. In *Proc. ISMM '13*, 109–118. <https://doi.org/10.1145/2464157.2466484>
- [21] Stephen M. Blackburn, Zixian Cai, Rui Chen, Xi Yang, John Zhang, and John Zigman. 2025. Rethinking Java Performance Analysis. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '25)*. <https://doi.org/10.1145/3669940.3707217>
- [22] Anthropic. 2026. Claude Code overview. <https://code.claude.com/docs/en/overview>
- [23] Anthropic. 2026. Introducing Claude Opus 4.6. <https://www.anthropic.com/news/claude-opus-4-6>
- [24] Anthropic. 2026. Building with extended thinking. <https://platform.claude.com/docs/en/build-with-claude/extended-thinking>
- [25] Luca Traini, Vittorio Cortellessa, Daniele Di Pompeo, and Michele Tucci. 2023. Towards effective assessment of steady state performance in Java software: Are we there yet? *Empirical Software Engineering* 28, 1. <https://doi.org/10.1007/s10664-022-10247-x>
- [26] Edd Barrett, Carl Friedrich Bolz-Tereick, Rebecca Killick, Sarah Mount, and Laurence Tratt. 2017. Virtual Machine Warmup Blows Hot and Cold. *Proceedings of the ACM on Programming Languages* 1, OOPSLA, Article 52. <https://doi.org/10.1145/3133876>

10 Appendix

10.1 biojava Extra Figures

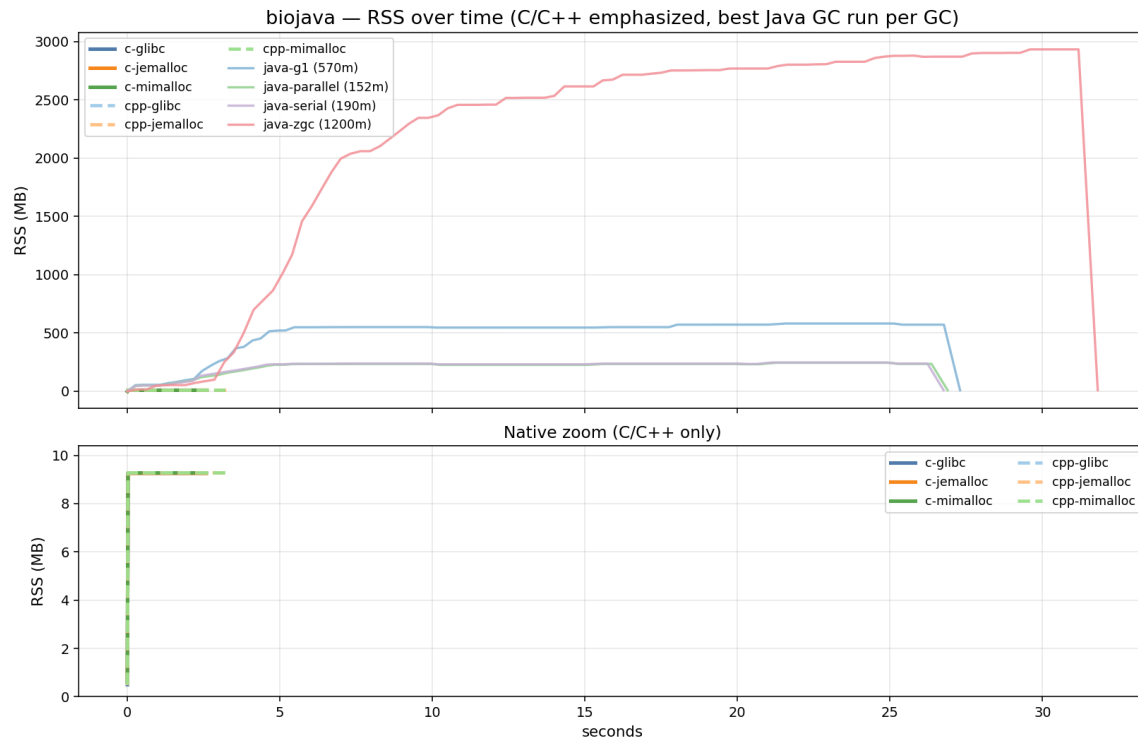


Figure 6: **biojava has much larger Java RSS over time than its native and C++ probe executions.** The Java configurations sustain longer memory footprints; it can be seen that the simplifications of the ports could be causing the C/C++ ports to execute artificially faster. The results do support the findings of RQ3, though the results should still be read cautiously.

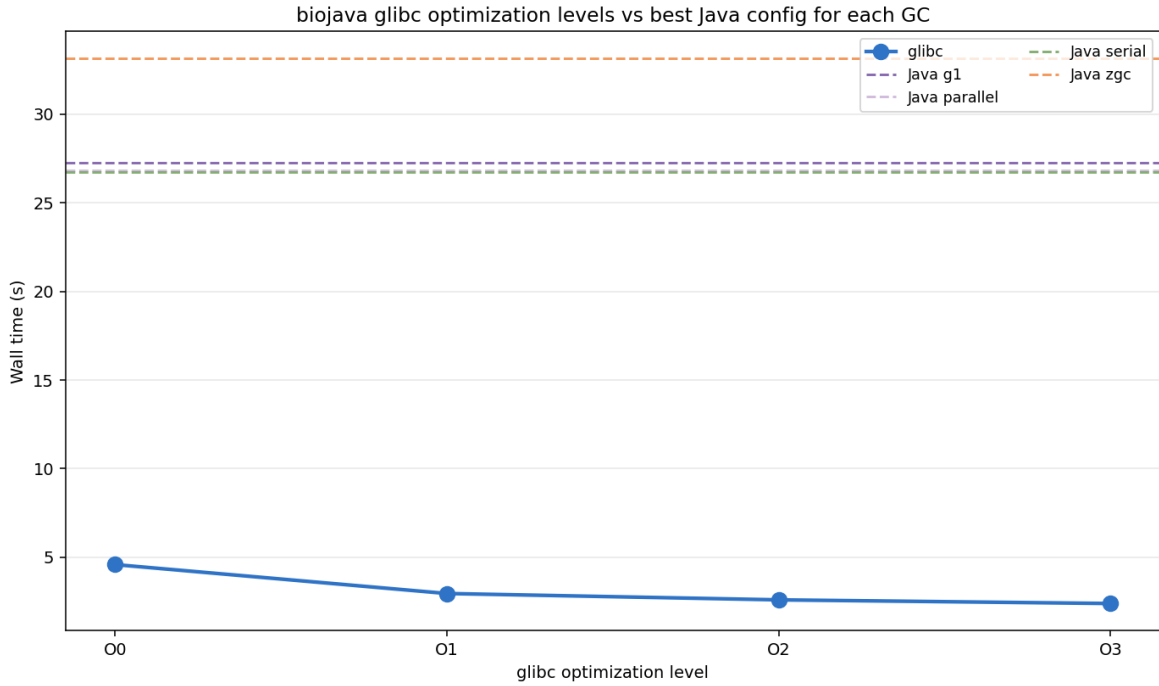


Figure 7: **Compiler optimization lowers the biojava glibc baseline but not by any meaningful amount.** The best Java configurations for each collector remain far above the native curve. The figure supports the RQ7 results that compiler optimization can improve the native ports' performance, but is not integral to the noted Java/native gap.

10.2 graphchi Extra Figures

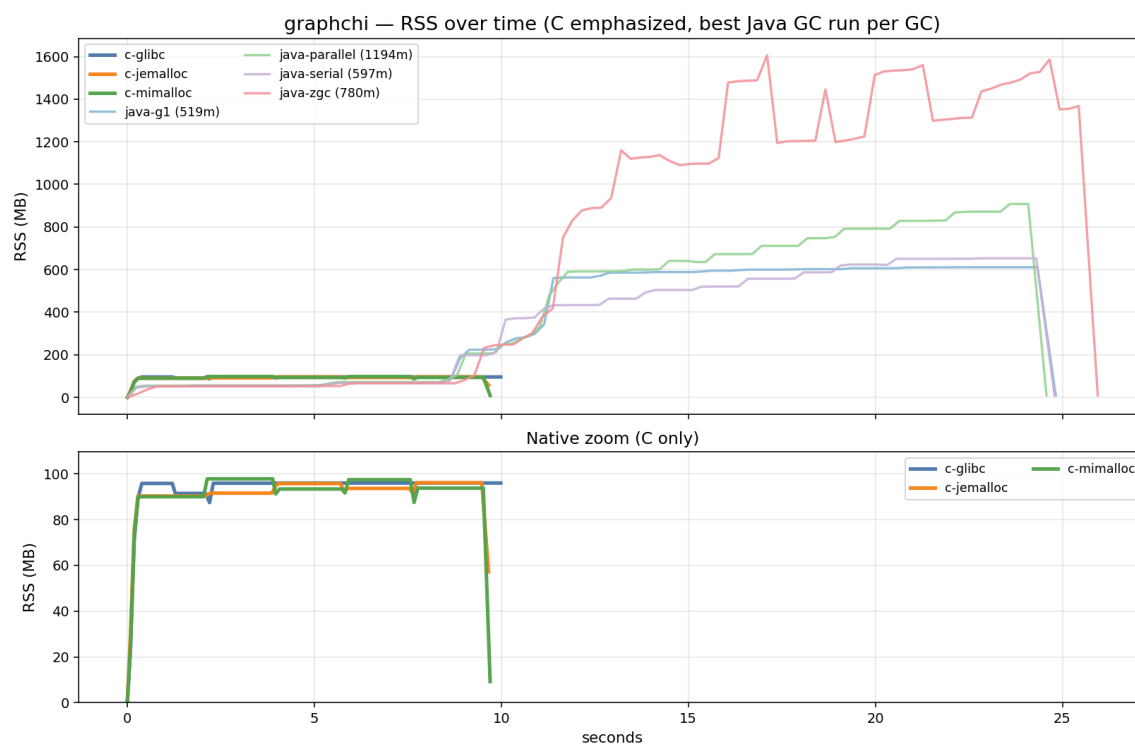


Figure 8: **RSS over time for graphchi follows the pattern of the Java memory over time, performing worse than the C ports.** The native C runs finish quickly and remain near the bottom of memory usage, while the Java configurations run longer and sustain much larger resident footprints. Interestingly, there seems to be a little more dynamic memory usage happening between the general-purpose allocators than we have seen before. The figure shows that the allocator choice changes the native footprint only slightly relative to the Java/native difference.

10.3 h2 Extra Figures



Figure 9: **RSS-over-time behavior for h2, following similar memory usage patterns that have been seen before.** The native C runs have higher peaks than G1 and Serial, but those runs execute far longer than the native baselines. This figure shows the unusual h2 case, where Java is much slower, even though the peak-RSS comparison is less one-sided than in other benchmarks. Thus, suggesting that EMM may not be as memory-efficient as GCs, but it still encourages faster execution.

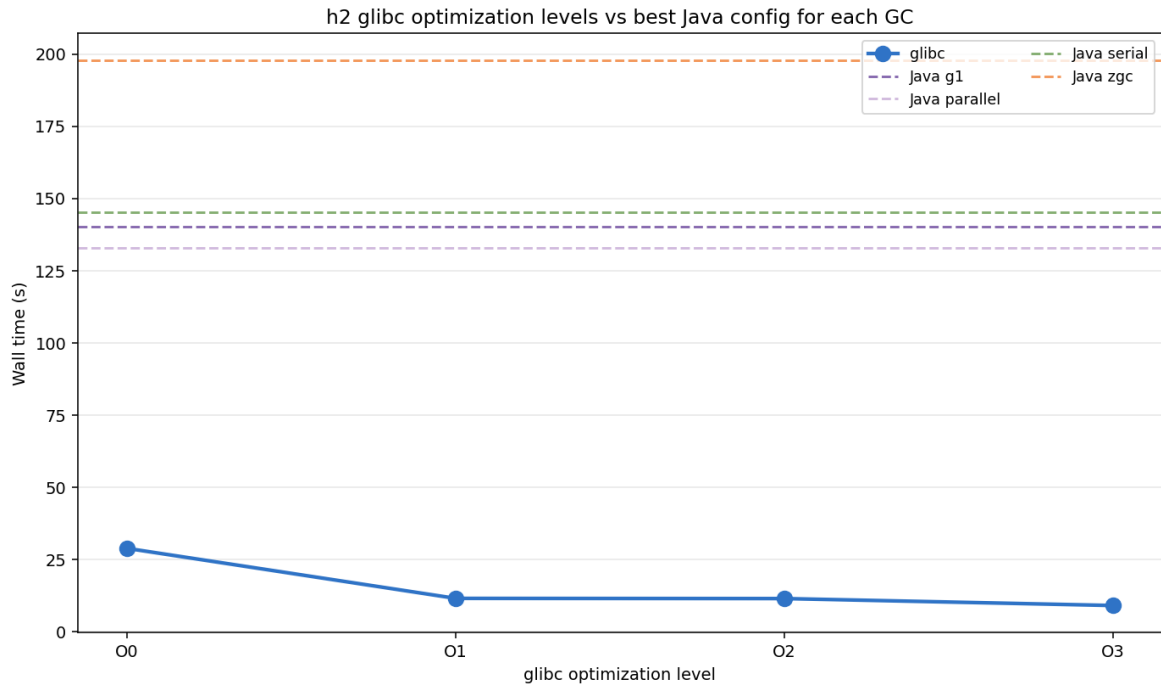


Figure 10: **Native compiler optimization substantially improves the h2 glibc baseline from O0, but quickly levels off, and the optimized native execution remains far below Java.** Runtime drops sharply when moving away from O0, after which the native curve stays well under the best Java collector lines. Once again, we see that different levels of optimization do not affect the overall Java/native gap.

10.4 lusearch Extra Figures

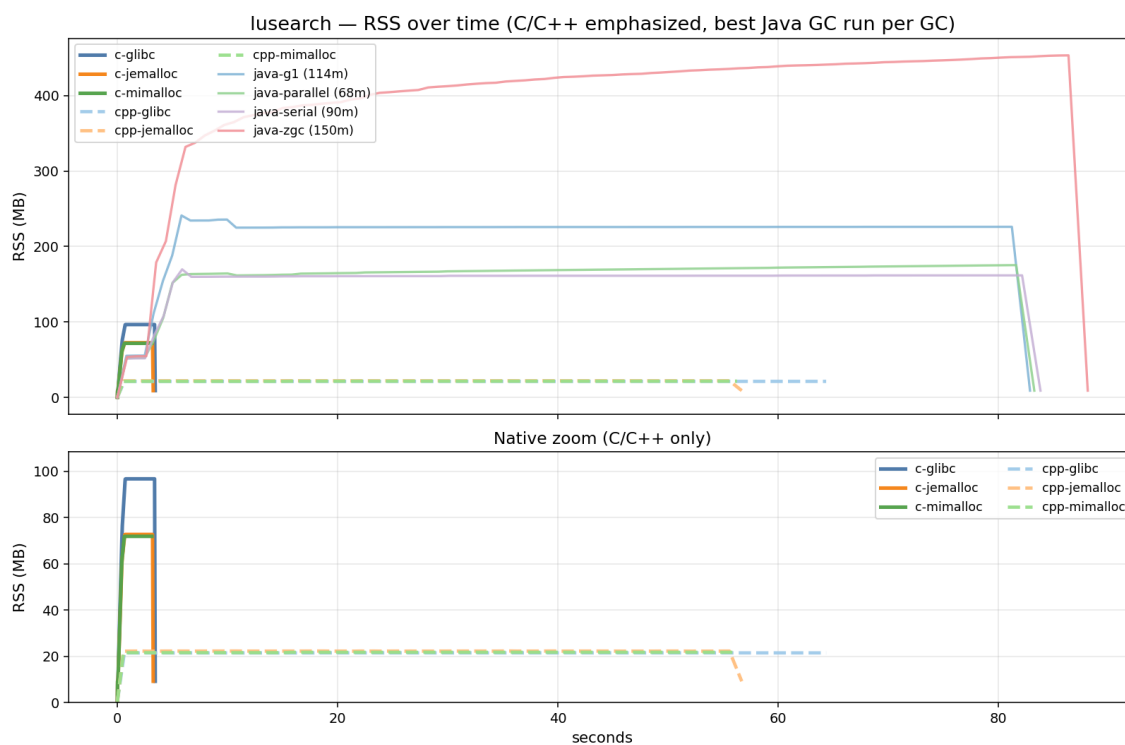


Figure 11: **lusearch** has low native RSS but much longer and higher Java RSS over time. The C ports finish quickly with a small resident footprint compared to the Java runs, but with a higher peak than the C++ ports. As noted before, Java sustains higher memory usage for longer, and C++, interestingly, follows a similar trajectory, albeit with lower memory usage and faster execution time. The graph supports the notion that looking at Resident memory alone can not fully explain the gap between Java/native.

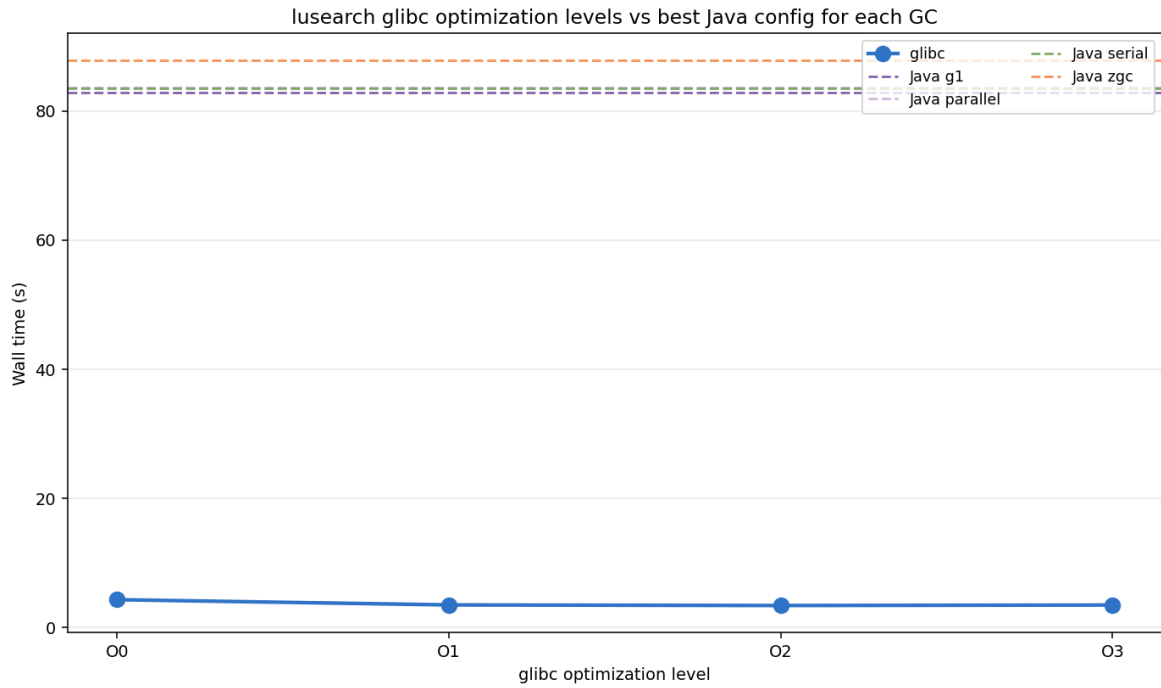


Figure 12: **Compiler optimization had little affect on lusearch glibc baseline.** The native runtime remains near the bottom of the plot across optimization levels. Once again, suggesting that, at least for these ports, optimization has little effect on EMM, as noted by the lack of improvement. The lusearch Java/native gap is driven more by implementation structure and Java-side work than by the native compiler setting.

10.5 sunflow Extra Figures

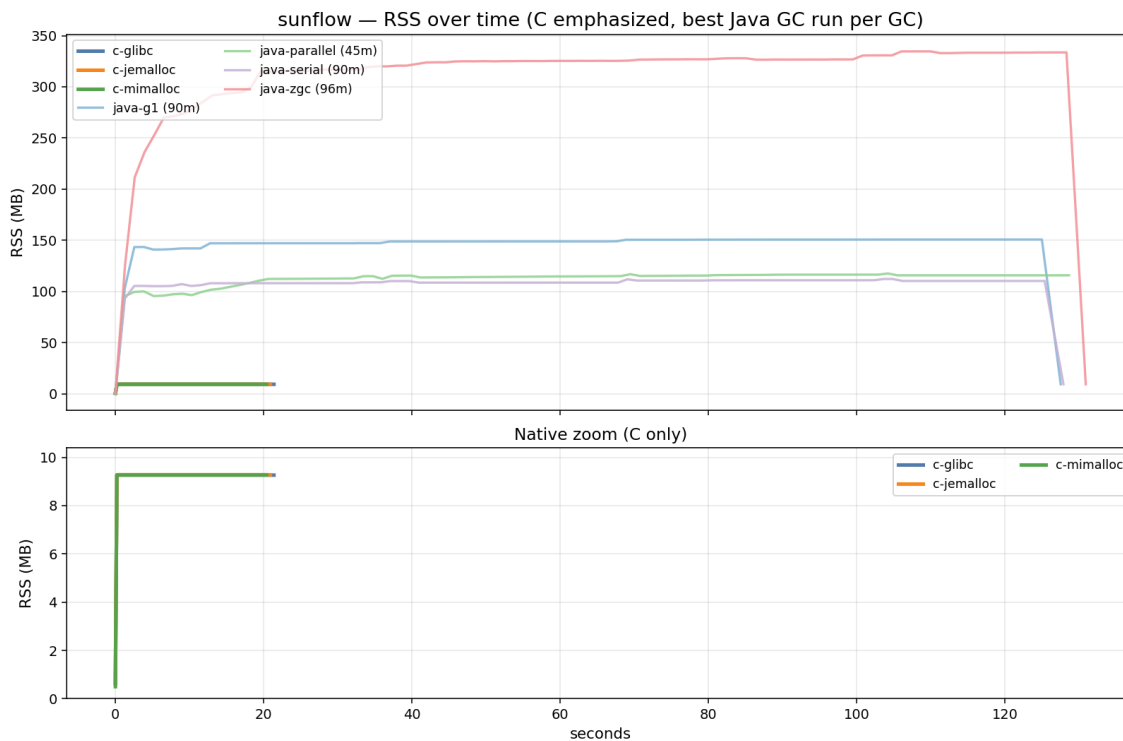


Figure 13: **sunflow** has a large and sustained memory-footprint separation between native and Java execution. The native C code does not pick up as much memory pressure as Java does, perhaps allowing it to execute faster. For this benchmark, the choice of allocator has little effect in C but a large effect in Java, with ZGC requiring the most memory.

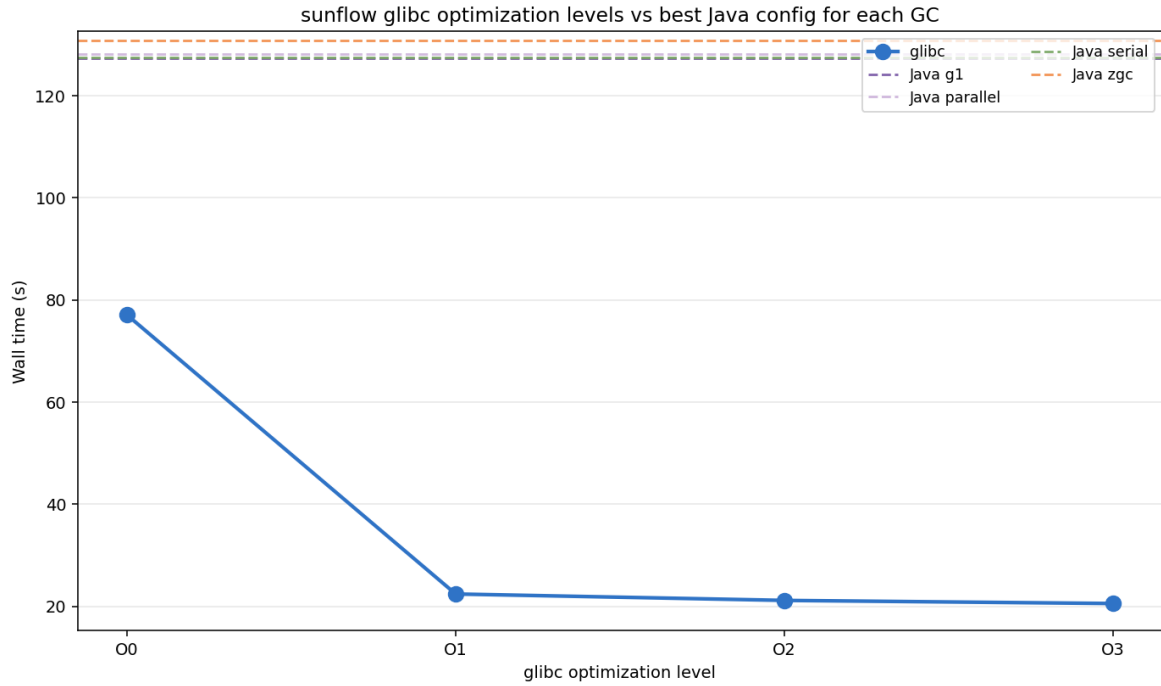


Figure 14: **Compiler optimization strongly improves the sunflow native baseline before the curve plateaus.** The largest gain appears when moving from O0 to optimized builds, after which native runtime remains far below the best Java runs. The figure supports the RQ7 result that optimization widens the measured gap relative to O0 but has no impact on the end pattern.

10.6 xalan Extra Figures



Figure 15: **xalan Java runs sustain much larger RSS than the native port.** The native runs finish quickly with lower memory usage, while Java configurations grow quickly and maintain higher memory pressure for most of the execution. The graph adheres to the results discussed in RQ3.

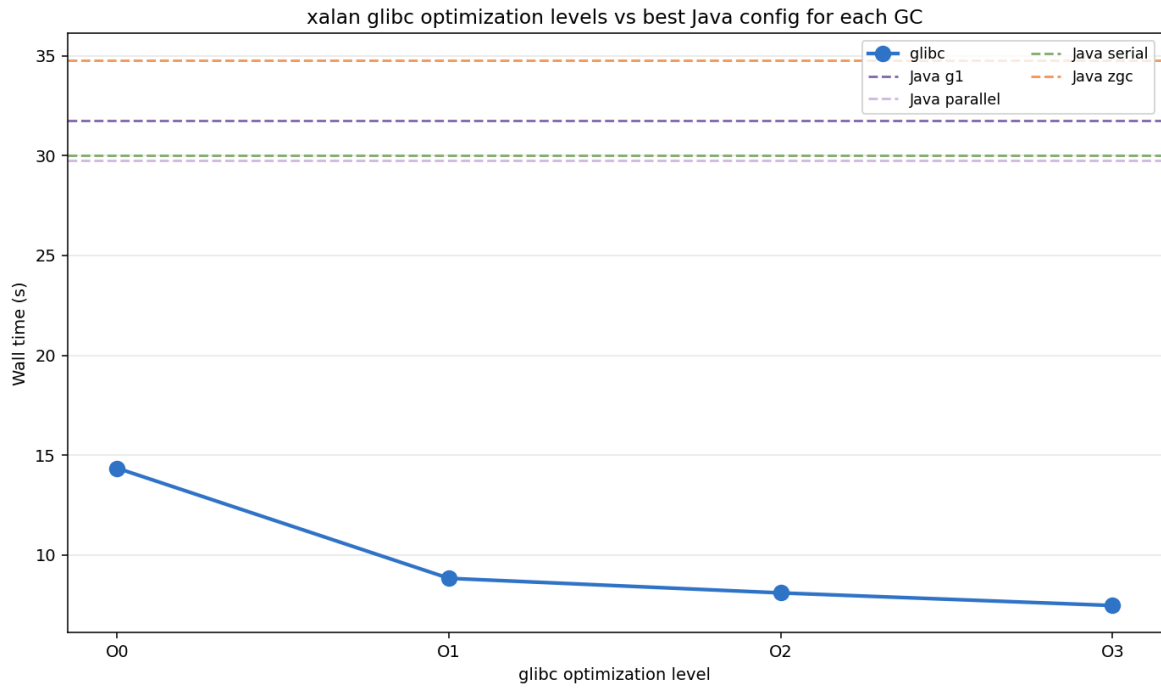


Figure 16: **xalan glibc native build improves as optimization levels increase with the baseline, while preserving that Java/native gap not changing in a meaningful way.**

10.7 zxing Extra Figures



Figure 17: **This figure shows why zxing should be treated as an anomaly rather than for EMM or GC.** Java runs sustain higher memory pressure; however, there are more noticeable peaks, valleys, and subtle increases across the GCs than before. As can be seen with the C allocator, there is high variability between them, which does not follow previous patterns. Thus, this further shows that zxing is an anomaly in our benchmark results.

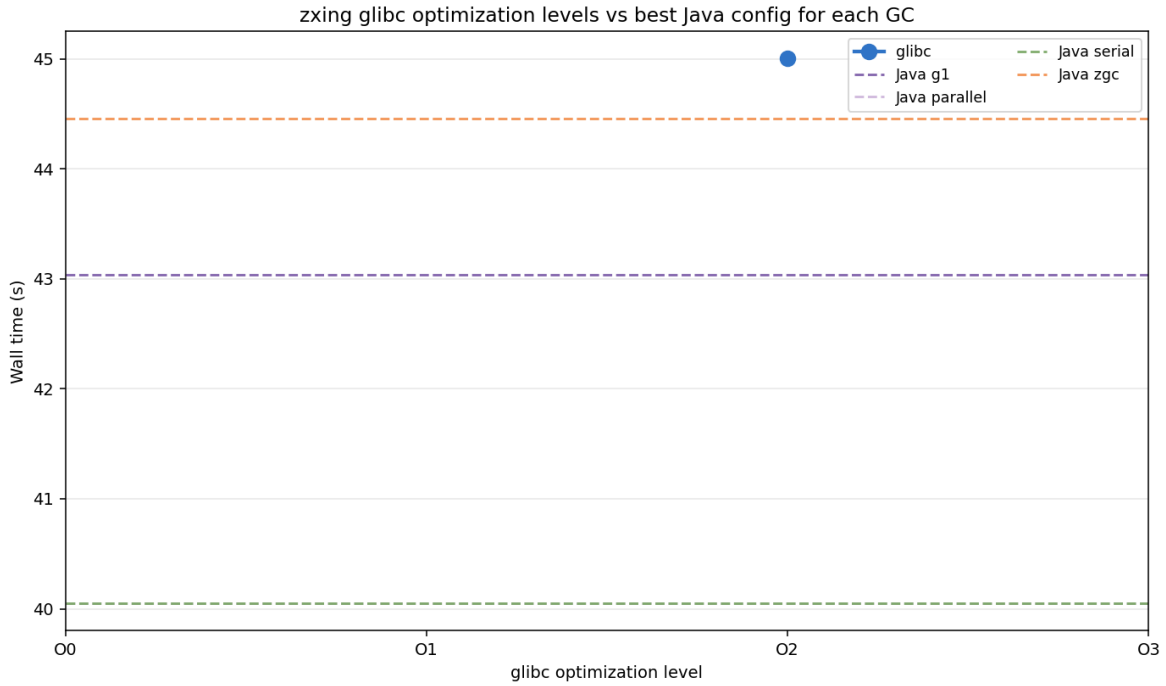


Figure 18: **This figure shows that the zxing glibc optimization data is incomplete and should not be used as a strong RQ7 case.** The data from multiple runs appears corrupted at different optimization levels, suggesting an inherent flaw in the C port of the benchmark. Thus suggesting that some ports an LLM may attempt to use may require further examination beyond validation checks.